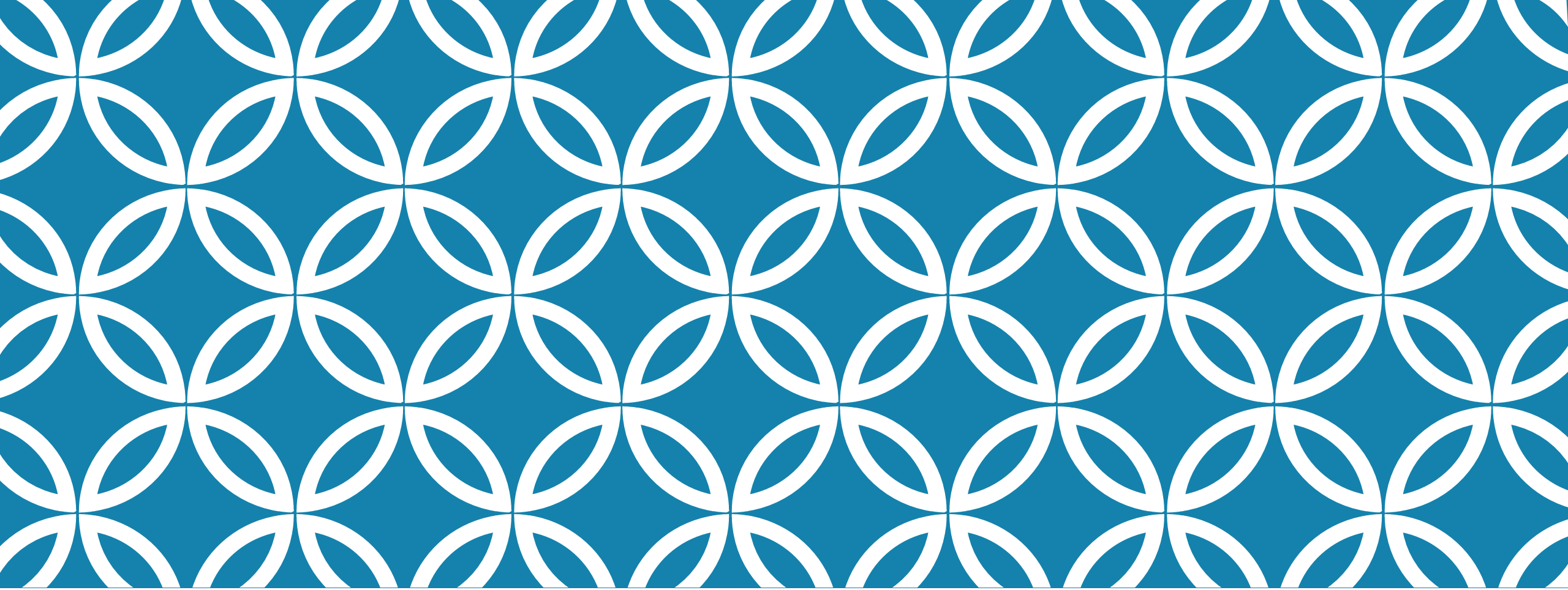


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

STRING SINIFININ METOT VE ÖZELLİKLERİ

- **Metot:**
  - Tekrar eden kod parçalarını kullanmayı kolaylaştırır.
  - Kodun okunabilirliğini ve anlaşılırlığını artırır.
- **Metotlar:**
  - Parametre alabilir veya almadan tanımlanabilir.
  - Bazı metotlar değer döndürürken bazıları döndürmez.

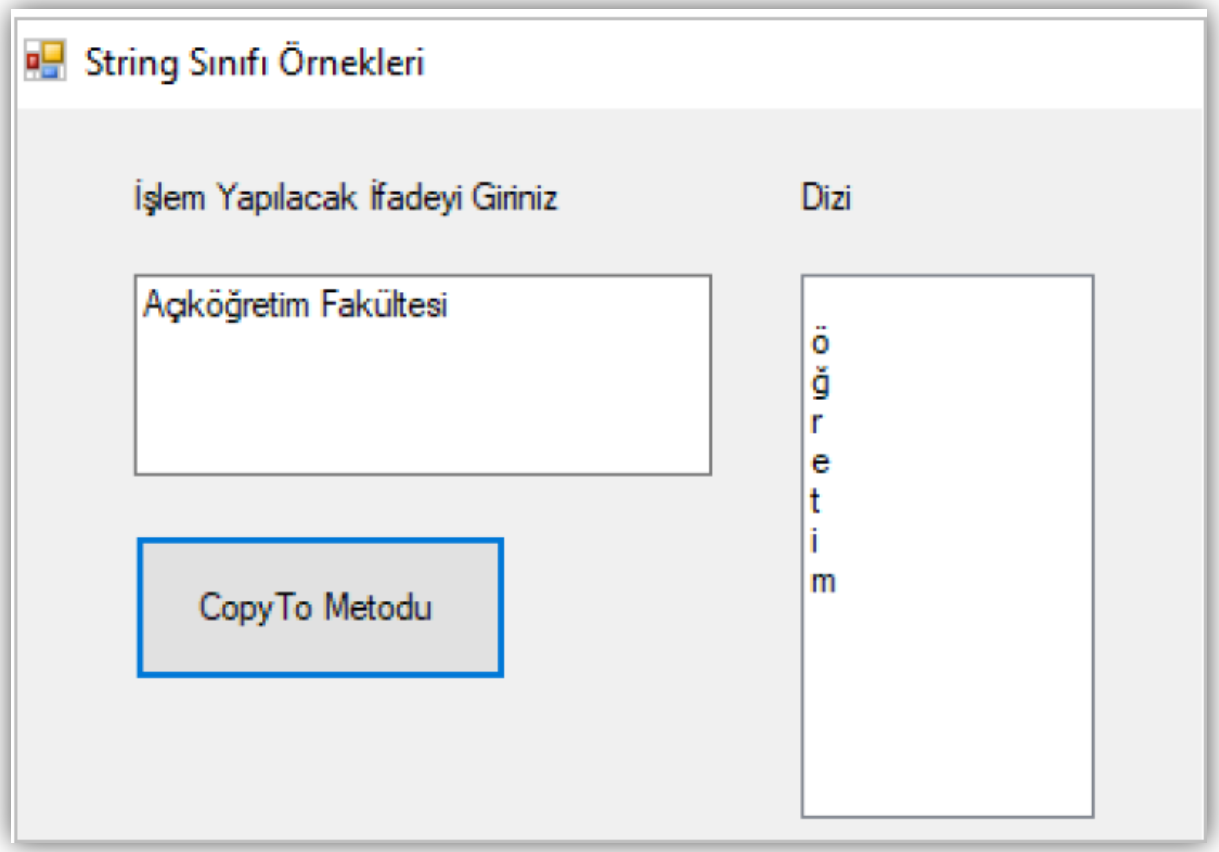
STRING SINIFININ METOT VE ÖZELLİKLERİ

- **C# String Sınıfı Metotları:**
  - Metin üzerinde kopyalama, değiştirme, birleştirme, parçalama ve karşılaştırma işlemleri yapılabilir.
- **Kopyalama ve Değiştirme Metotları**
- **CopyTo() Metodu:**
  - Bir string ifadeyi belirli bir konumdan itibaren bir char dizisine kopyalar.

STRING SINIFININ METOT VE ÖZELLİKLERİ

- **CopyTo() Kullanım Örneği:**

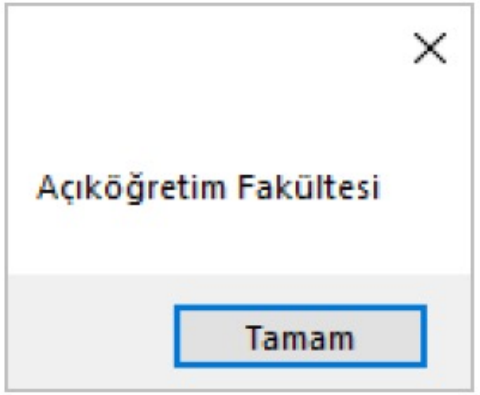
```
private void CopyToMetot_Click(object sender, EventArgs e)
{
        string ifade = textBox1.Text;
        char[] dizi = new char[20];
        ifade.CopyTo(4, dizi, 1, 7);
        foreach(char eleman in dizi)
        {
                listBox1.Items.Add(eleman);
        }
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **Clone()**
  - String bir değerin başka bir değişkene atanmasını sağlar.

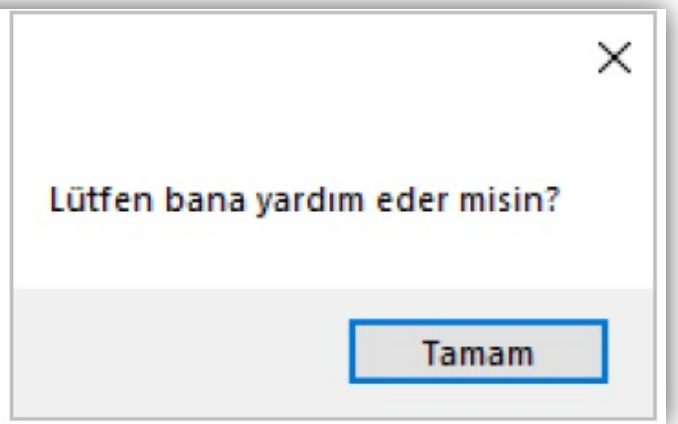
```
private void CloneMetot_Click(object sender, EventArgs e)
{
      string ifade = "Açıköğretim Fakültesi";
      string kopya= (string)ifade.Clone();
       MessageBox.Show(kopya);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **Insert()**
  - String bir ifadeye belirli bir konumundan itibaren yeni bir string eklemeyi sağlar.
  - Bu fonksiyon metinin başına, sonuna veya ortasına yeni bir ifade eklenmek istendiğinde kullanılır.

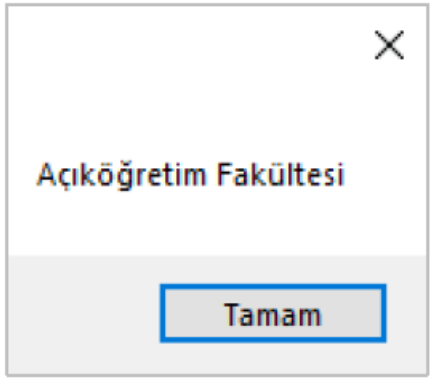
```
private void InsertMetot_Click(object sender, EventArgs e)
{
        string cumle= "bana yardım eder misin";
        string yeniCumle = cumle.Insert(0, "Lütfen ");
        MessageBox.Show(yeniCumle);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **Replace()**
  - String bir ifadeyi değiştirmek için kullanılır. İki parametre alır.
  - Verilen ilk parametreyi ifade içinde arar ve bulduğunda bu değeri ikinci parametredeki değer ile değiştirir

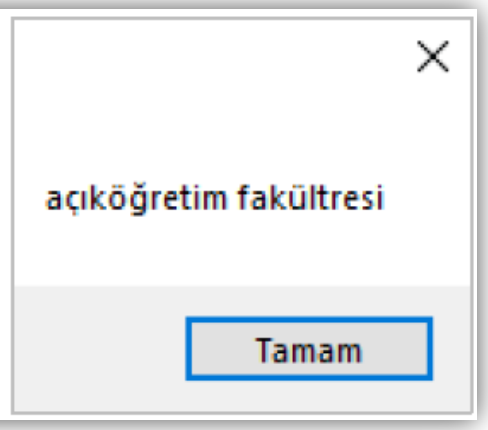
```
private void ReplaceMetot_Click(object sender, EventArgs e)
{
    string ifade = "Açıkoğretim Fakültesi";
    string yenilfade = ifade.Replace("o", "ö");
    MessageBox.Show(yenilfade);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **ToLower()**
  - Verilen string bir ifadedeki tüm karakterleri küçük harfe dönüştürür.

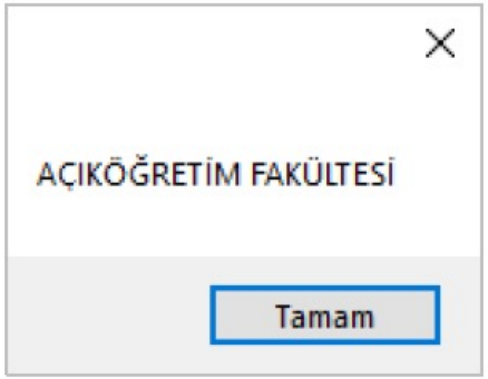
```
private void ToLowerMetot_Click(object sender, EventArgs e)
{
        string ifade = "Açıköğretim Fakültesi";
        string yenilfade = ifade.ToLower();
        MessageBox.Show(yenilfade);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **ToUpper()**
  - Verilen string bir ifadedeki tüm karakterleri büyük harfe dönüştürür.

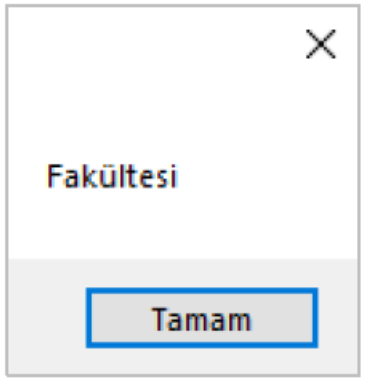
```
private void ToUpperMetot_Click(object sender, EventArgs e)
{
      string ifade = "açıköğretim fakültesi";
       string yenilfade = ifade.ToUpper();
       MessageBox.Show(yenilfade);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **Substring()**
  - Verilen string ifadeyi alt string ifadelere bölmeyi sağlar. İki ya da tek parametre alabilir. İki parametre almış ise ilk parametre ifadenin kaçıncı indisten sonra bölüneceğini, ikinci parametre ise alt stringin uzunluğunu belirtir. Eğer tek bir parametre girilmiş ise o indisten itibaren ifadenin sonuna kadar bölme işlemi yapılır.

```
private void SubstringMetot_Click(object sender, EventArgs e)
{
        string ifade = "Açıköğretim Fakültesi";
        string sonuc = ifade.Substring(11);
        MessageBox.Show(sonuc);
}
```



STRING SINIFININ METOT VE ÖZELLİKLERİ

- **ToCharArray()**

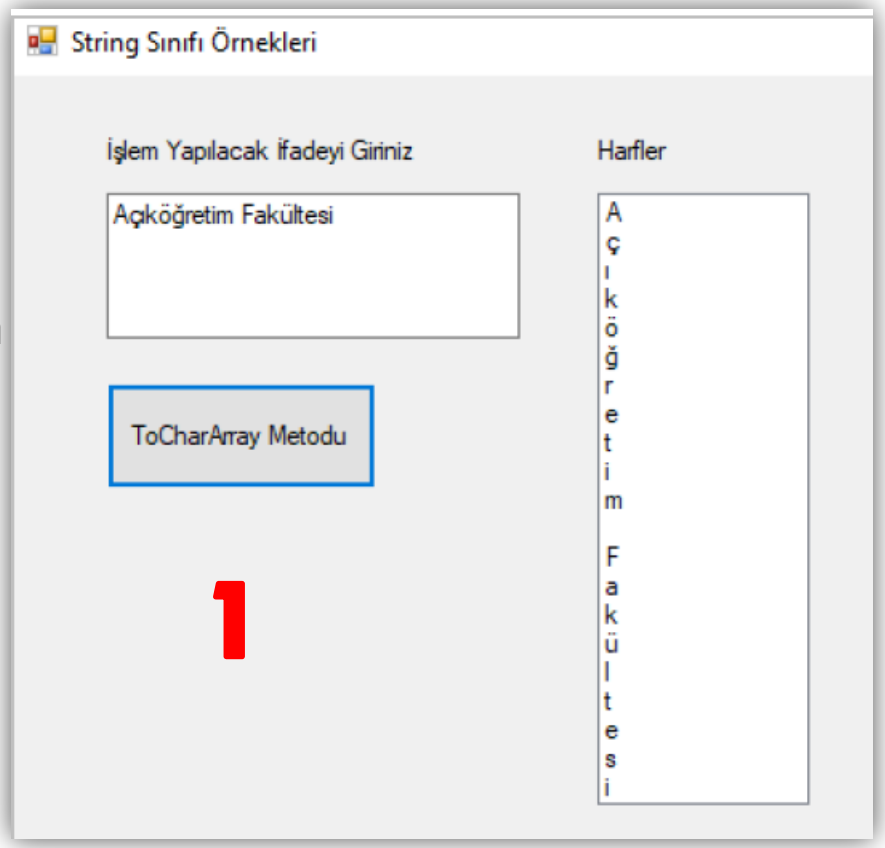
1. Verilen string ifadeyi karakterlere ayırıp char tipinde tanımlanmış bir diziye aktarır.
2. İlk parametre kaçıncı indexten itibaren karaktere dönüştürüleceğini gösterirken; ikinci parametre kaç karakterin işleme alınacağını gösterir.

2

```
string ifade = textBox1.Text;
char[] harfler = ifade.ToCharArray(4,8);
foreach (char harf in harfler)
{
        listBox1.Items.Add(harf);
}
```

1

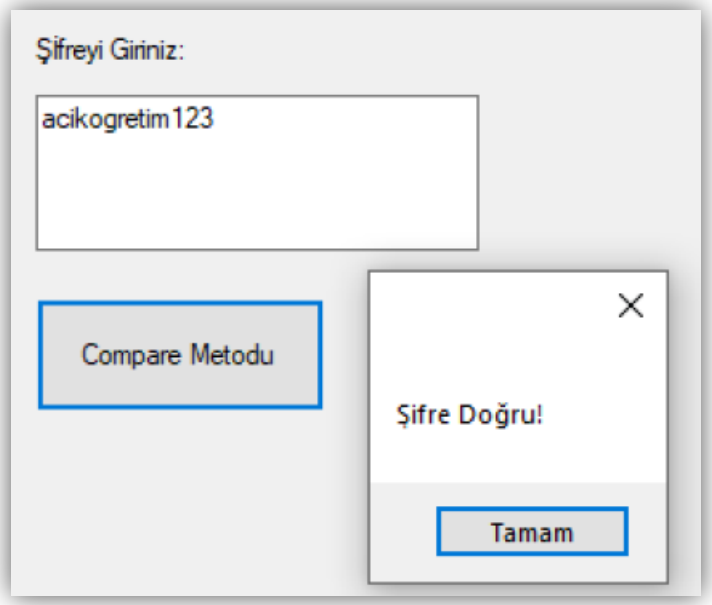
```
string ifade = textBox1.Text;
char[] harfler = ifade.ToCharArray();
foreach (char harf in harfler)
{
        listBox1.Items.Add(harf);
}
```



KARŞILAŞTIRMA METOTLARI

- İki string ifadeyi karşılaştırmak, alfabedeki önceliğini bulmak için kullanılan metotlardır.
- **Compare():** String iki ifadeyi karşılaştırmak için kullanılan metotlardan biridir

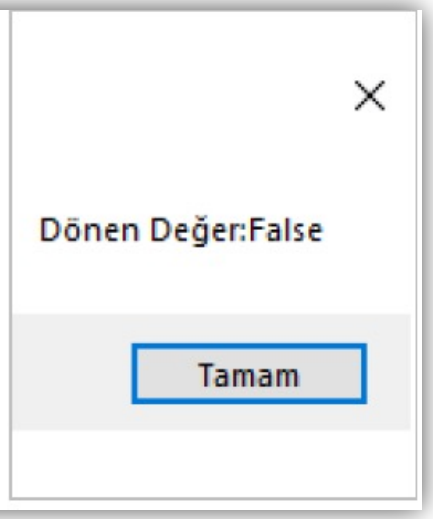
```
string sifre = "acikogretim123";
string girilenSifre = textBox1.Text;
int sonuc = string.Compare(sifre, girilenSifre);
if(sonuc==0)
{
        MessageBox.Show("Şifre Doğru!");
}
else
        MessageBox.Show("Şifre Yanlış!");
```



KARŞILAŞTIRMA METOTLARI

- **Equals():** Eğer iki string ifade birbirine eşit ise *true*, değilse *false* değeri döner.

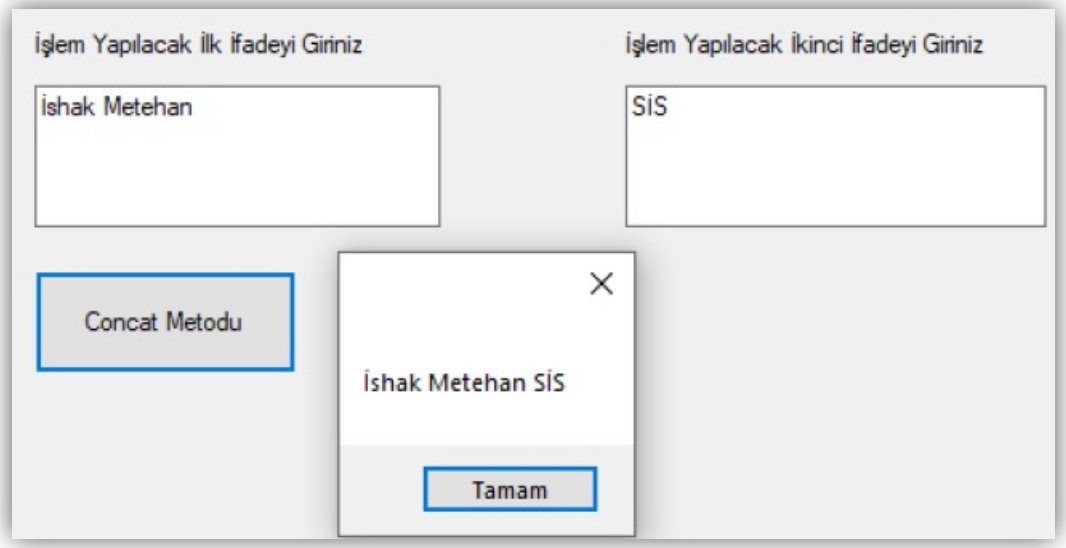
```
private void EqualsMetot_Click(object sender, EventArgs e)
{
    string ifade1 = "ankara";
    string ifade2 = "Ankara";
    bool sonuc = string.Equals(ifade1, ifade2);
    //Harf duyarlılığı olduğuna dikkat ediniz.
    MessageBox.Show("Dönen Değer:" + sonuc.ToString());
}
```



BİRLEŞTİRME VE PARÇALAMA METOTLARI

- Birden fazla string ifadeyi birleştirmek veya verilen bir string ifadeyi parçalara ayırmak istenildiğinde kullanılan metotlardır.
- **Concat():** Verilen parametreleri birleştirerek geriye tek bir değer döndürür. Yani string ifadeler üzerinde ekleme işlemi yapar. İkide fazla parametre alabilir.

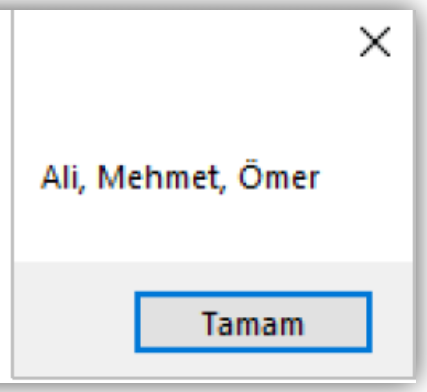
```
string ifade1 = textBox1.Text;
string ifade2 = textBox2.Text;
string yeni = string.Concat(ifade1, ifade2);
MessageBox.Show(yeni);
```



BİRLEŞTİRME VE PARÇALAMA METOTLARI

- **Join():** String türünde verilen bir dizideki elemanları birleştirerek tek bir değişkende toplar.

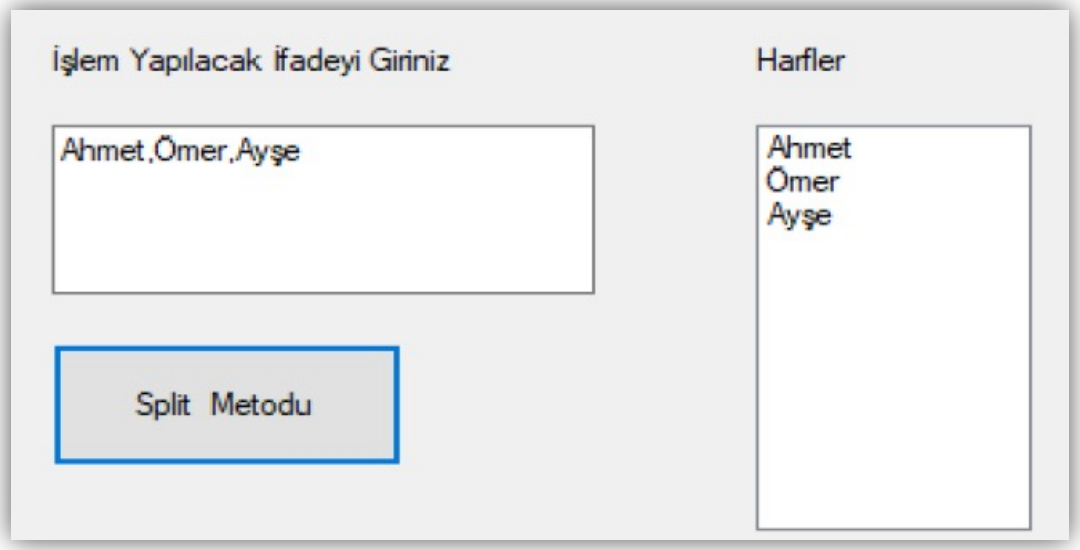
```
private void JoinMetot_Click(object sender, EventArgs e)
{
        string[] dizi = { "Ali", " Mehmet", " Ömer "};
        string yeni= string.Join(",", dizi);
        MessageBox.Show(yeni);
}
```



BİRLEŞTİRME VE PARÇALAMA METOTLARI

- **Split():** String türünde verilen bir ifadeyi belirlenen ayraca göre parçalayıp **string** türündeki bir diziye atar.

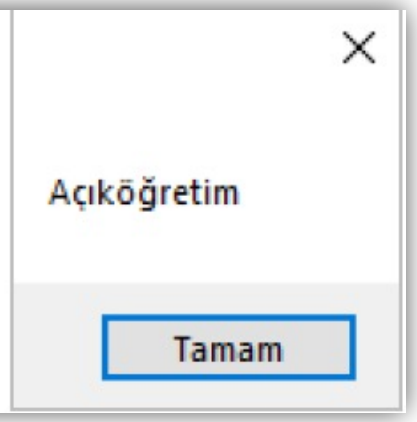
```
string ifade = textBox1.Text;
string[] dizi = ifade.Split(',');
foreach(string kelime in dizi)
{
        listBox1.Items.Add(kelime);
}
```



BİRLEŞTİRME VE PARÇALAMA METOTLARI

- **Remove():** Verilen string ifadedeki karakterleri silmeyi sağlar.

```
private void RemoveMetot_Click(object sender, EventArgs e)
{
        string ifade = "Açıköğretim Fakültesi";
        string yeniifade = ifade.Remove(12,9);
        MessageBox.Show(yeniifade);
}
```



ARAMA METOTLARI

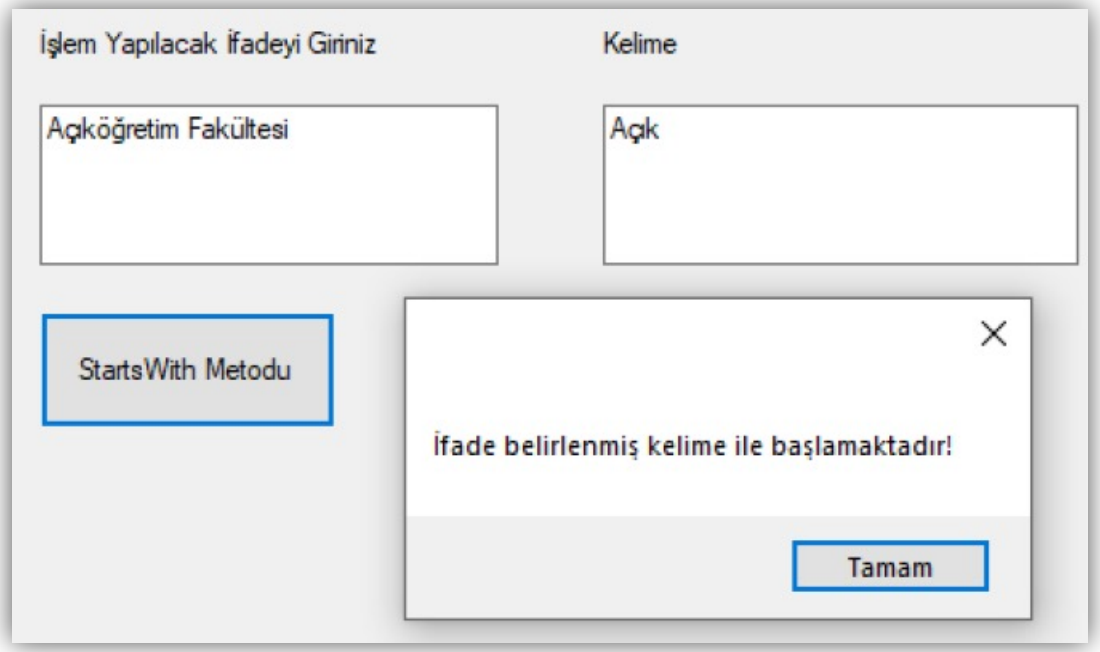
- Verilen string ifadede de başka bir ifadeyi arayan, aranan kelime bulunduğunda konumunu gösteren metotlardır.
- **IndexOf():** String bir ifade içinde herhangi bir karakter ya da başka bir string ifadeyi arar. Aranan değer bulunursa bulunan indexi geri döndürür. Arana değer, ifade içinde yoksa -1 değerini döndürür.

```
string ifade = textBox1.Text;
string arananKelime = textBox2.Text;
int sonuc = ifade.IndexOf(arananKelime);
if (sonuc != -1)
{
        MessageBox.Show("Aranan kelimenin bulunduğu index: " + sonuc.ToString());
}
else
        MessageBox.Show("Aranan kelime bulunmamaktadır!");
```

ARAMA METOTLARI

- **StartsWith():** Bir string ifadenin verilen parametre değeri ile başlayıp başlamadığını kontrol eden metottur. Eğer verilen parametre ile başlanıyorsa geriye true yoksa false değeri döner

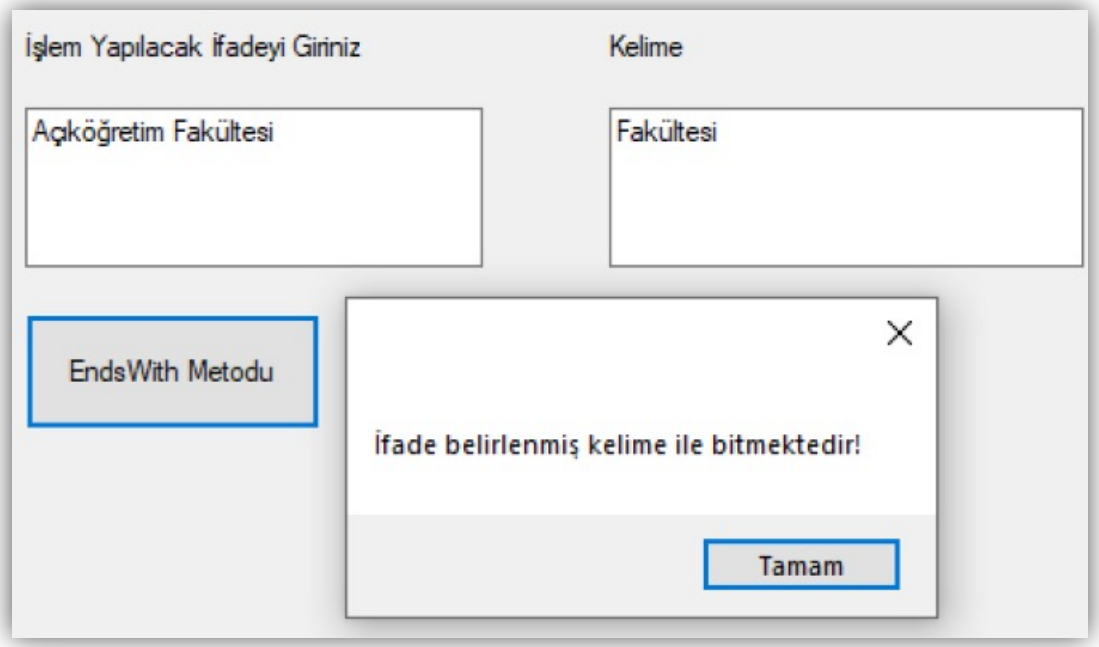
```
string ifade = textBox1.Text;
string baslayanKelime = textBox2.Text;
bool sonuc = ifade.StartsWith(baslayanKelime);
if (sonuc==true)
{
        MessageBox.Show("İfade belirlenmiş kelime ile başlamaktadır!");
}
else
        MessageBox.Show("İfade belirlenmiş kelime ile başlamamaktadır!");
```



ARAMA METOTLARI

- **EndsWith():** Bir string ifadenin verilen parametre değeri ile bitip bitmediğini kontrol eden metottur. Eğer verilen parametre ile bitiyorsa geriye *true* yoksa *false* değeri döner.

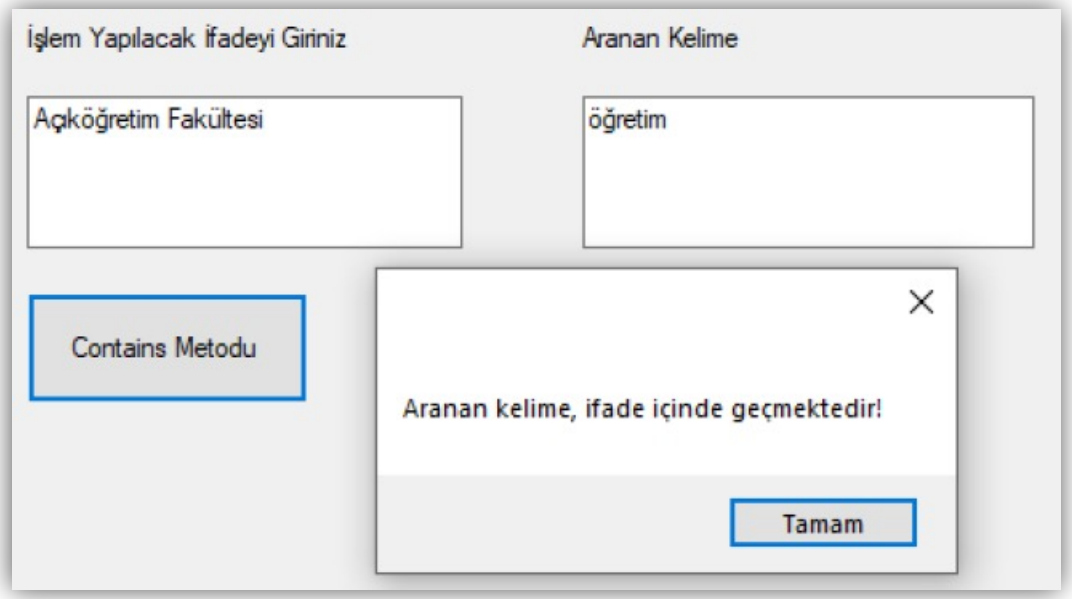
```
string ifade = textBox1.Text;
string baslayanKelime = textBox2.Text;
bool sonuc = ifade.EndsWith(baslayanKelime);
if (sonuc==true)
{
        MessageBox.Show("İfade belirlenmiş kelime ile bitmektedir!");
}
else
        MessageBox.Show("İfade belirlenmiş kelime ile bitmemektedir!");
```



ARAMA METOTLARI

- **Contains():** Verilen string ifade içinde gönderilen parametreyi arar. Eğer gönderilen parametre ifade içinde geçiyorsa *true* değeri döner.

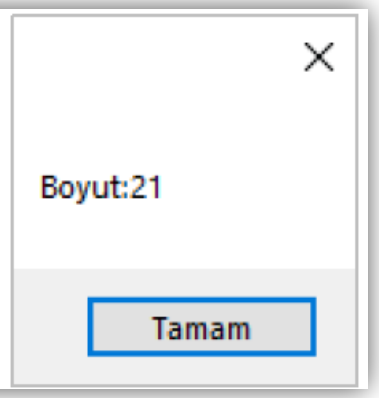
```
string ifade = textBox1.Text;
string arananKelime = textBox2.Text;
bool sonuc = ifade.Contains(arananKelime);
if (sonuc == true)
{
        MessageBox.Show("Aranan kelime, ifade içinde geçmektedir!");
}
else
        MessageBox.Show("Aranan kelime bulunmamaktadır!");
```



BOYUT BİLGİSİ İLE İLGİLİ METOTLAR

- String ifadenin kaç karakterden oluştuğunu, boyut bilgisini veren metotlardır.
- **Length:** Dizilerde kullanılan ve dizinin boyutunu veren bir metottur. String bir ifade için kullanıldığında ise ifadedeki karakter sayısını verir.

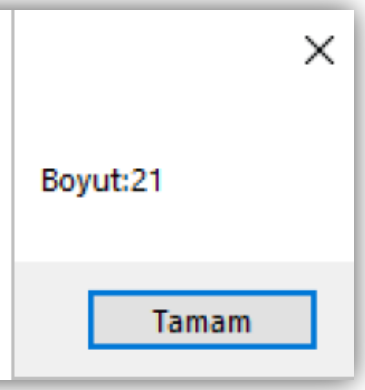
```
private void LengthMetot_Click(object sender, EventArgs e)
{
        string ifade = "Açıköğretim Fakültesi";
        int boyut = ifade.Length;
        MessageBox.Show("Boyut:"+boyut.ToString());
}
```



BOYUT BİLGİSİ İLE İLGİLİ METOTLAR

- **Count():** Length metodu ile aynı işlevi yapar. String ifadedeki karakter sayını verir.

```
private void CountMetot_Click(object sender, EventArgs e)
{
    string ifade = "Açıköğretim Fakültesi";
    int boyut = ifade.Count();
    MessageBox.Show("Boyut:"+boyut.ToString());
}
```



BOYUT BİLGİSİ İLE İLGİLİ METOTLAR

- **IsNullOrEmpty():** Verilen string ifadenin içeriğini kontrol eden bir metottur. İçerik boşsa ya da null değer var ise true değeri yoksa false değeri döner.

```
string ifade = textBox1.Text;
bool sonuc = string.IsNullOrEmpty(ifade);
if(sonuc==true)
{
        MessageBox.Show("Lütfen bir değer giriniz!");
}
```